

Theory: 2D BarDistribution, mixture MALC, optimal-transport aggregation, and variance reduction

0.1 Two-dimensional BarDistribution head

Training is asking how plausible each location is at every query. Then looking up how tall the predicted surface is there, and penalising when it's short.

What the head predicts. For every query covariate X , given the observational context $\mathcal{D}_{\text{obs}} = \{(X_i, T_i, Y_i)\}_{i=1}^N$, the head outputs a full probability density over the pair of potential outcomes under both treatment values:

$$f(y_0, y_1 \mid X, \mathcal{D}_{\text{obs}}) : \mathbb{R}^2 \rightarrow [0, \infty), \quad \iint_{\mathbb{R}^2} f(y_0, y_1 \mid X, \mathcal{D}_{\text{obs}}) dy_0 dy_1 = 1. \quad (1)$$

Every point in $\mathbb{R}^2$ has a defined density value; nothing is excluded. This is an extension of the one-dimensional head, which outputs $f(y \mid X, \mathcal{D}_{\text{obs}})$ over a single outcome $y \in \mathbb{R}$.

Why the joint, not two marginals. A pair of marginal densities $\hat{p}_0(y_0 \mid X, \mathcal{D}_{\text{obs}})$ and $\hat{p}_1(y_1 \mid X, \mathcal{D}_{\text{obs}})$ recovers the true joint only if $Y_{\text{do}(0)} \perp Y_{\text{do}(1)} \mid X$ — an assumption that fails on essentially every real DGP, where the potential outcomes share exogenous noise.

Preprocessing: rescaling outcomes to $[-1, 1]$. Before the head is applied, every Y_i in the context is min-max rescaled by the context Y -range so it lies in $[-1, 1]$:

$$\tilde{Y}_i = 2 \cdot \frac{Y_i - Y_{\min}}{Y_{\max} - Y_{\min}} - 1, \quad Y_{\min} = \min_i Y_i, \quad Y_{\max} = \max_i Y_i. \quad (2)$$

This is standard input normalisation so a single interior grid of fixed width is meaningful across queries, and so the tail scales $\sigma_d^{(\ell)}$ below are comparable across queries. The choice of $[-1, 1]$ (as opposed to, say, $[0, 1]$) is a convention. The scale is inverted at test time to report outcomes in the original units.

For the rest of this subsection y_0, y_1 refer to the rescaled scale, so context observations lie in $[-1, 1]$.

Why f must cover all of $\mathbb{R}^2$, not just $[-1, 1]^2$. At training time we cannot guarantee that a labelled paired outcome (y_0^*, y_1^*) from the DGP will fall inside the rescaling box $[-1, 1]^2$ — rescaling in (??) is done on the *context* Y -values, and the target held-out outcome can easily land outside the observed context range. If f assigns zero density to any labelled target, the training loss $-\log f(y_0^*, y_1^* \mid X, \mathcal{D}_{\text{obs}})$ diverges to $+\infty$ and the model becomes untrainable. So f must place strictly positive density everywhere in $\mathbb{R}^2$. This is the constraint that forces the piecewise construction below. However, we would still expect most of the training prediction to fall under the $[-1, 1]^2$ range.

Three-part construction. A plain discrete grid over $[-1, 1]^2$, with one probability per bin, is flexible enough to capture the complexity of our datasets, but it cannot be extended to all of $\mathbb{R}^2$ without requiring infinitely many parameters. Restricting the grid to a bounded region keeps the parameter count finite, but it also forces the predicted density to vanish outside the box. What we need, then, is a component that assigns non-zero density over the whole of $\mathbb{R}^2$ while still being described by finitely many parameters — a Gaussian being the natural candidate. On its own, however, a Gaussian is too rigid to fit multi-modal or skewed joints. We therefore combine the two, which partitions $\mathbb{R}^2$ into three kinds of region. Each tail is a half-normal anchored at the boundary: a normal density centred at $+1$ or -1 , restricted to the side lying outside $\mathcal{I}$ and renormalised. Anchoring at the boundary removes the location parameter (μ), leaving the scale σ as the only free quantity, and σ admits a direct reading — it tells you how far your model should predict fall past the boundary. If true values leave $[-1, 1]$ so only marginally, as with 1.02 or -1.05 , the likelihood favours a small σ ; if they routinely land far outside, a larger σ is favoured. Learning σ is therefore the act of learning how far the true distribution extends beyond the modelled region, and it assigns a corresponding amount of probability mass there. We prefer to use a Gaussian tail for three reasons: (1) it is normalisable on the half-line with a closed-form constant, (2) it has a closed-form CDF; (3) its log-density is quadratic, so gradients with respect to both the observation and σ are smooth throughout the domain. Heavier-tailed alternatives such as a truncated Student- t are equally tractable but introduce a further shape parameter that is weakly identified when few observations fall outside $\mathcal{I}$; the Gaussian tail proved sufficient on our datasets.

The following provides the three parts:

1. *Interior* $\mathcal{I} := [-1, 1]^2$: a $K \times K$ grid of bins, with one learned probability per bin. Gives a piecewise-constant density that can approximate any joint shape in the bulk.
2. *Edge* (one coordinate outside $\mathcal{I}$, one inside): a half-Gaussian tail along the outside coordinate, multiplied by the grid’s boundary conditional along the inside coordinate. Four edges (one per axis-direction pair) continuously extend the grid past $\mathcal{I}$ in each single direction.
3. *Corner* (both coordinates outside $\mathcal{I}$): a bivariate normal truncated to the appropriate quadrant, with the correlation inherited from the interior grid. Four corners extend the density into the four diagonal directions.

These nine regions (one interior, four edge, four corner) partition $\mathbb{R}^2$. The head’s density is a weighted mixture over the nine, which is strictly positive everywhere by construction.

What the head learns. For each query the transformer emits a fixed-size vector of $K^2 + 9 + 4$ real numbers: K^2 interior-bin logits, 9 region-weight logits, and 4 raw tail scales (one per axis-direction pair). Standard link functions (softmax on the first two groups, softplus on the third, detailed below) turn these into the interior probabilities p_{j_0, j_1} , the region weights w_r , and the tail scales $\sigma_d^{(\ell)}$ that together determine the density f . Training is a single objective: for each labelled paired outcome (y_0^*, y_1^*) , minimise the negative log-density $-\log f(y_0^*, y_1^* \mid X, \mathcal{D}_{\text{obs}})$. Gradient descent flows through the softmax and softplus link functions back into the transformer’s parameters, which learn to push more density toward observed outcomes at each step. No auxiliary losses.

The rest of this subsection formalises each piece: the interior bin structure, the per-query output that parameterises everything, the density expression in each of the three region types, how the tail correlation is read from the interior grid rather than trained separately, and finally the unified mixture density and loss.

Figures/nine_region_partition_of_R2.png

Figure 1: Caption

Interior bin structure. $\mathcal{I} = [-1, 1]^2$ is partitioned into a uniform $K \times K$ grid of square bins of width $\delta := 2/K$. Bins are indexed by $(j_0, j_1) \in \{0, 1, \dots, K-1\}^2$; bin B_{j_0, j_1} is the $\delta \times \delta$ square with lower-left corner $(-1 + j_0\delta, -1 + j_1\delta)$. Under this convention a rescaled coordinate $y \in [-1, 1]$ belongs to bin index

$$j_\ell(y) := \lfloor (y + 1)/\delta \rfloor, \quad (3)$$

which appears throughout the density expressions below.

Per-query output. For each query X , the transformer emits $K^2 + 9 + 4$ real numbers, grouped into three sets. Each set is transformed into a valid density parameter with a standard link function that guarantees positivity:

- *Interior logits* $\ell \in \mathbb{R}^{K^2}$, one per bin, softmaxed into a discrete joint on $\mathcal{I}$:

$$p_{j_0, j_1}(X, \mathcal{D}_{\text{obs}}) = \frac{\exp(\ell_{j_0, j_1})}{\sum_{i_0, i_1} \exp(\ell_{i_0, i_1})}, \quad \sum_{j_0, j_1} p_{j_0, j_1} = 1. \quad (4)$$

The softmax gives every bin strictly positive probability, so no bin can zero out the density on any target.

- *Region logits* $u \in \mathbb{R}^9$, one per region, softmaxed into the nine mixture weights:

$$w_r(X, \mathcal{D}_{\text{obs}}) = \frac{\exp(u_r)}{\sum_{s=1}^9 \exp(u_s)}, \quad r \in \{1, \dots, 9\}, \quad \sum_r w_r = 1. \quad (5)$$

These control how much mass the model allocates to the interior vs. the four edges vs. the four corners.

- *Raw tail parameters* $(\sigma_L^{(0)\text{raw}}, \sigma_R^{(0)\text{raw}}, \sigma_L^{(1)\text{raw}}, \sigma_R^{(1)\text{raw}})$, one per axis $\ell \in \{0, 1\}$ and direction $d \in \{L, R\}$, passed through softplus with a small floor $\varepsilon > 0$:

$$\sigma_d^{(\ell)}(X, \mathcal{D}_{\text{obs}}) = \delta \cdot (\text{softplus}(\sigma_d^{(\ell)\text{raw}}) + \varepsilon). \quad (6)$$

Softplus guarantees $\sigma > 0$ so the tail Gaussians below are well-defined; the δ multiplier makes tail widths and interior bin widths comparable so the interior–tail seam is smooth in scale.

The full density, one region at a time. The density $f(y_0, y_1 \mid X, \mathcal{D}_{\text{obs}})$ is defined piecewise depending on where (y_0, y_1) falls relative to the box $\mathcal{I}$. Below we describe each of the three cases; the full mixture is stated in (??).

Case 1 — interior ($y_0, y_1 \in [-1, 1]$). The discrete grid (??) defines a piecewise-constant density on $\mathcal{I}$: mass p_{j_0, j_1} is spread uniformly over the area δ^2 of bin B_{j_0, j_1} , giving density $p_{j_0, j_1}/\delta^2$ inside that bin:

$$f_{\text{inner}}(y_0, y_1 \mid X, \mathcal{D}_{\text{obs}}) = \frac{p_{j_0(y_0), j_1(y_1)}}{\delta^2}. \quad (7)$$

Both the bin assignment $j_\ell(\cdot)$ in (??) and the softmax weights in (??) are differentiable in the logits ℓ , so gradients flow back through this density to the transformer.

Case 2 — edge (one coordinate outside). When exactly one of y_0, y_1 falls outside $[-1, 1]$, we need a density that (i) smoothly extends off the boundary in the outside coordinate, and (ii) still reflects where along the boundary the point sits in the inside coordinate. Take the representative subcase $y_0 \in [-1, 1]$ and $y_1 < -1$ (the other three edges are symmetric). We factor

$$f_{\text{mix}}(y_0, y_1 \mid X, \mathcal{D}_{\text{obs}}) = p_L^{(1)}(y_1) \cdot g(y_0 \mid y_1 = -1), \quad (8)$$

where the tail contribution

$$p_L^{(1)}(y_1) = \frac{2}{\sigma_L^{(1)}} \phi\left(\frac{y_1 - (-1)}{\sigma_L^{(1)}}\right) \quad (9)$$

is a half-Gaussian on $(-\infty, -1]$ centred at the boundary -1 with scale $\sigma_L^{(1)}$ (ϕ is the standard normal density, the factor 2 makes the half-normal integrate to one on the half-line), and

$$g(y_0 \mid y_1 = -1) = \frac{p_{j_0(y_0), 0}}{(\sum_{i_0} p_{i_0, 0}) \cdot \delta} \quad (10)$$

is the discrete boundary conditional obtained by evaluating p on the bottom row (bins with $j_1 = 0$, i.e., adjacent to $y_1 = -1$) and renormalising over the in-range axis so the density integrates to one along y_0 . The approximation $p(y_0 \mid y_1 < -1) \approx p(y_0 \mid y_1 = -1)$ is discussed in §?? below. All four edge subcases apply the same construction with the boundary and axis adjusted accordingly.

Case 3 — corner (both coordinates outside). When both y_0 and y_1 fall outside $[-1, 1]$, the point lies in one of four diagonal quadrants determined by signs $(s_0, s_1) \in \{L, R\}^2$. We use a truncated bivariate normal on that quadrant, centred at the corresponding corner $(c_0, c_1) \in \{-1, +1\}^2$, scaled by the tail widths $(\sigma_{s_0}^{(0)}, \sigma_{s_1}^{(1)})$ from (??):

$$f_{\text{tail}}(y_0, y_1 \mid X, \mathcal{D}_{\text{obs}}) = \frac{1}{Z(s_0, s_1, \rho)} \phi_2(\tilde{y}_0, \tilde{y}_1; \rho), \quad \tilde{y}_\ell = \frac{y_\ell - c_\ell}{\sigma_{s_\ell}^{(\ell)}}, \quad (11)$$

where $\phi_2(\cdot, \cdot; \rho)$ is the standard bivariate normal density with correlation ρ , and the denominator Z is the total probability mass the bivariate normal assigns to that quadrant — the correct normaliser to make f_{tail} integrate to one on the quadrant. This mass is given in closed form by Sheppard's identity (?):

$$Z(s_0, s_1, \rho) = \begin{cases} \frac{1}{4} + \frac{\arcsin \rho}{2\pi} & \text{if } s_0 = s_1, \\ \frac{1}{4} - \frac{\arcsin \rho}{2\pi} & \text{if } s_0 \neq s_1. \end{cases} \quad (12)$$

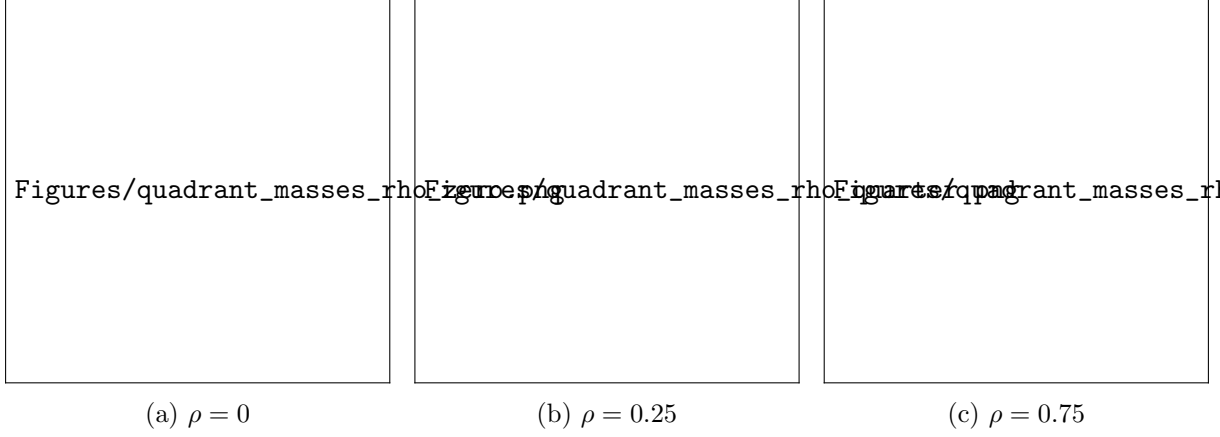


Figure 2: Quadrant masses of the bivariate normal centred at the corner $(1,1)$. Correlation shifts mass onto the diagonal, so the truncation constant Z is $1/4$ only at $\rho = 0$.

To demonstrate this normalization step by Z , consider the top-right corner, that is the region where $y_0 > 1$ and $y_1 > 1$, on which a bivariate normal centred at $(1,1)$ is placed. A bivariate normal is supported on the entire plane, so centring it at $(1,1)$ also assigns density to points such as $(0.9, 1.2)$, $(1.3, 0.8)$ and $(0.5, 0.4)$, none of which lie in the corner region: they belong to the top edge, the right edge and the interior respectively, each of which is already modelled by its own component. Left uncorrected, the corner component would therefore claim mass that has already been accounted for elsewhere, and the nine regional densities would not combine into a single normalised density on $\mathbb{R}^2$.

The correlation ρ used here is not a separate learned parameter; it is derived directly from the interior grid, as described next.

Correlation of the tails inherited from the interior grid. The correlation ρ in (??)–(??) is defined as the Pearson correlation of the discrete distribution $\{p_{j_0, j_1}\}$ evaluated at bin midpoints $m_\ell(j_\ell) := -1 + \delta(j_\ell + \frac{1}{2})$:

$$\begin{aligned} \bar{y}_\ell &= \sum_{j_0, j_1} p_{j_0, j_1} m_\ell(j_\ell), \\ v_\ell &= \sum_{j_0, j_1} p_{j_0, j_1} (m_\ell(j_\ell) - \bar{y}_\ell)^2, \\ c &= \sum_{j_0, j_1} p_{j_0, j_1} (m_0(j_0) - \bar{y}_0)(m_1(j_1) - \bar{y}_1), \\ \rho &= \frac{c}{\sqrt{v_0 v_1} + \varepsilon}. \end{aligned} \tag{13}$$

Two reasons for deriving ρ this way rather than training a separate scalar. First, it removes one degree of freedom, so the model cannot learn a ρ for the tails that disagrees with what the interior grid already implies. Second, whatever coupling (Y_0, Y_1) shows in the interior bulk is exactly what governs the tails, so the density is coherent across the seam between $\mathcal{I}$ and its complement.

Full mixture and training loss. The joint density at any point in $\mathbb{R}^2$ is the mixture of the nine region components weighted by (??):

$$f(y_0, y_1 \mid X, \mathcal{D}_{\text{obs}}) = w_1 f_{\text{inner}} + \sum_{r=2}^5 w_r f_{\text{mix}}^{(r)} + \sum_{r=6}^9 w_r f_{\text{tail}}^{(r)}, \tag{14}$$

where the four $f_{\text{mix}}^{(r)}$ correspond to the four edge orientations and the four $f_{\text{tail}}^{(r)}$ to the four corners. Because the nine regions partition $\mathbb{R}^2$, at any query point exactly one region is active and only its component contributes to f .

Given a labelled paired outcome (y_0^*, y_1^*) , the training loss is the negative log-density

$$\mathcal{L}(y_0^*, y_1^* | X, \mathcal{D}_{\text{obs}}) = -\log f(y_0^*, y_1^* | X, \mathcal{D}_{\text{obs}}), \quad (15)$$

averaged over queries in a batch. Standard backpropagation flows gradients through the softmax and softplus link functions back into the transformer weights, which learn to push more density towards the observed (y_0^*, y_1^*) at each step. There is no auxiliary loss.

Notes on the edge-case approximation. The edge density (??) factors as $p(y_1) p(y_0 | y_1)$ and supplies the second factor by the boundary conditional $p(y_0 | y_1 = -1)$, holding it fixed for all $y_1 < -1$. The approximation is exact when $Y_0 \perp Y_1$, since the conditional is then constant in y_1 . Under dependence it displaces the conditional by an amount that grows with both the strength of dependence and the distance past the boundary: for a bivariate normal with correlation ρ and tail scale σ , the resulting excess negative log-likelihood per edge observation is approximately $\rho^2 \sigma^2 / \pi(1 - \rho^2)$.

This displacement is not something the edges could avoid by inheriting ρ in the way the corner components do. In a corner both coordinates are Gaussian, so dependence is carried by a single scalar and the conditional mean shifts automatically with distance from the corner. An edge instead couples a Gaussian tail to a histogram, for which no such scalar exists: letting the boundary conditional drift would require an explicit model of how the bin probabilities deform with y_1 , whose parameters would have to be estimated from the few observations falling outside $\mathcal{I}$. We therefore accept a bounded bias in place of that variance, and retain a density that is differentiable and evaluable in $\mathcal{O}(1)$ per query.

0.2 Multimodal density recovery via mixture MALC

The training-time head outputs the discrete grid $\{p_{j_0, j_1}\}$ of Eq. (??), which is a piecewise-constant density on $\mathcal{I}$. To recover a smooth density we build on the mean-adjusted log-concave (MALC) estimator of ? : MALC produces a shape-constrained nonparametric density estimate from grouped (binned) probabilities without bandwidth selection. Because a single log-concave density is unimodal by construction, we extend MALC to a K -component mixture in order to represent the multimodal joints that arise in causal inference under posterior uncertainty over compatible SCMs. We refer to ? for the theoretical properties of the estimator and its behaviour under grid refinement. We denote a single MALC component $\hat{f} \in \mathcal{F}_{\text{LC}}$.

Mixture MALC (K components). Because $\mathcal{F}_{\text{LC}}$ is unimodal, a single MALC fit cannot represent the multimodal joints that arise when the posterior concentrates on multiple compatible SCMs (Section ??). We therefore consider the mixture family

$$\mathcal{F}_{\text{MALC}}^{(K)} = \left\{ f = \sum_{k=1}^K \pi_k f_k : \pi \in \Delta^{K-1}, f_k \in \mathcal{F}_{\text{LC}} \ \forall k \right\}, \quad (16)$$

where each f_k is a single-component MALC estimator obtained by the procedure above. Membership in $\mathcal{F}_{\text{MALC}}^{(K)}$ permits up to K distinct modes while retaining the shape constraint within each component.

Mixture EM. For a fixed $K > 1$, components are combined via expectation–maximisation on the binned data. Let $\gamma_{j_0 j_1, k}^{(t)}$ denote the responsibility of component k for bin (j_0, j_1) at iteration t , and let $F_k^{(t)}(j_0, j_1) := \int_{B_{j_0, j_1}} f_k^{(t)}$ denote the bin-integrated component density. The two updates are

$$\gamma_{j_0 j_1, k}^{(t+1)} = \frac{\pi_k^{(t)} F_k^{(t)}(j_0, j_1)}{\sum_j \pi_j^{(t)} F_j^{(t)}(j_0, j_1)}, \quad (17)$$

$$\pi_k^{(t+1)} = \sum_{j_0, j_1} p_{j_0, j_1} \gamma_{j_0 j_1, k}^{(t+1)}, \quad (18)$$

$$f_k^{(t+1)} = \text{single-component refit on } \{p_{j_0, j_1} \gamma_{j_0 j_1, k}^{(t+1)}\}. \quad (19)$$

Initialisation is a Voronoi assignment around modal peaks detected on p (with a deterministic augmentation when K exceeds the number of detected peaks); the iterations stop after five consecutive non-improving log-likelihoods.

Algorithmic summary. Algorithm ?? collects the mixture EM iterations described above and the BIC-based order selection described below into a single pipeline.

Output. Once K^* is chosen, the final estimator is

$$\hat{p}_{\text{MALC}}(y_0, y_1) = \sum_{k=1}^{K^*} \hat{\pi}_k^* \hat{f}_k^*(y_0, y_1), \quad (20)$$

a smooth density on the convex hull of the synthetic points, evaluated at any query point $(y_0, y_1) \in \mathbb{R}^2$. Point estimates follow directly: $\hat{\mu}_{\text{mean}} = \int (y_1 - y_0) d\hat{p}_{\text{MALC}}$ and the 1D projection onto the diagonal $\tau = y_1 - y_0$ produces the treatment-effect distribution used for OT aggregation in Section ?. As $n \rightarrow \infty$, the absolute difference between $\hat{\mu}_{\text{mean}}$ and the true mean is bounded by δ . Since we set $K = 100$, this difference is below 0.02 as n increases to infinity.

Theoretical guarantees for the single-component MALC estimator (from which each $\hat{f}_k^*$ is drawn) are established in ?; here we treat the mixture as a modelling device and do not claim additional properties for $\hat{p}_{\text{MALC}}$ beyond those inherited componentwise.

0.3 Population aggregation via the 2-Wasserstein barycenter

To turn per-query treatment-effect distributions into a single population-level estimate we use the 2-Wasserstein barycenter. Let $\hat{p}_i(\tau)$ denote the MALC-smoothed per-query treatment-effect density (Section ??) for query X_i with cumulative distribution function F_i and quantile function $Q_i := F_i^{-1}$. Given nonnegative weights $\alpha_1, \dots, \alpha_N$ summing to 1, the (weighted) 2-Wasserstein barycenter is

$$p^* = \operatorname{argmin}_q \sum_{i=1}^N \alpha_i W_2^2(\hat{p}_i, q), \quad W_2^2(f, g) = \int_0^1 (F_f^{-1}(t) - F_g^{-1}(t))^2 dt, \quad (21)$$

where W_2 is the 1D 2-Wasserstein distance in its quantile form. In one dimension this problem admits a closed-form solution: for measures on $\mathbb{R}$ with finite second moments, the barycenter (??) exists uniquely and its quantile function is the weighted average of the input quantile functions (?),

$$Q^*(t) = \sum_{i=1}^N \alpha_i Q_i(t), \quad t \in (0, 1). \quad (22)$$

Figures/ihdp_n10_ot.png

Figure 3: OT aggregation on IHDP realisation $r=0$ with $N=200$ context units. Faded curves: per-query $p(\tau \mid X_i)$ from the MALC-smoothed joint. Solid green: 2-Wasserstein barycenter. Vertical lines: true population ATE, OT-mode, OT-mean, and the naive mean of per-query means.

Implementation. Because Q^* is defined explicitly by (??), no iterative optimisation is needed. Given the per-query densities on a common grid, Algorithm ?? computes the barycenter in three quantile passes and one density-recovery pass. All steps are $\mathcal{O}(N \cdot M)$ vector arithmetic; the barycenter is thus much cheaper than the per-query MALC fits it aggregates.

Illustration on IHDP. Figure ?? shows the aggregation on IHDP realisation $r=0$ with $N=200$ context observations and $N=10$ test queries. The ten faded per-query densities peak around the individual τ values that the model has inferred for each unit; the barycenter (solid green) inherits their shape while collapsing to a single population summary. The barycenter mode $\widehat{\text{ATE}}_{\text{OT-mode}} = 0.58$ is meaningfully closer to the true population ATE 0.63 than either the naive mean-of-means 0.47. Mean-based estimator underestimates because the per-query densities are right-skewed with heavier right shoulders than left, whereas the barycenter mode is invariant to this skew.

OT-mode and OT-mean point estimates. From p^* we extract two point estimates of the population average treatment effect $\text{ATE} = \mathbb{E}_X[\tau(X)]$:

$$\widehat{\text{ATE}}_{\text{OT-mode}} = \operatorname{argmax}_{\tau} p^*(\tau), \quad \widehat{\text{ATE}}_{\text{OT-mean}} = \int \tau p^*(\tau) d\tau. \quad (23)$$

Both are functionals of p^* and hence deterministic given the per-query densities. Here purpose is more to estimate the density rather than improving the point-wise estimation, but due to nature of skewness in the benchmark datasets, it also improves the results.

Algorithm 1 Mixture MALC from binned probabilities $\{p_{j_0, j_1}\}$.

Require: Binned joint $\{p_{j_0, j_1}\}$ on a $K_g \times K_g$ grid; maximum order $K_{\max}$; synthetic-sample size B ; tolerance Π .

Ensure: Multimodal density estimate $\hat{p}_{\text{MALC}} = \sum_k \hat{\pi}_k^* \hat{f}_k^*$.

```

1: function FITCOMPONENT( $w_{j_0, j_1}$ ) ▷ single MALC on a weighted grid
2:   return log-concave MLE (??) on  $\{y_b\}$ 
3: end function
4: for  $K \in \{1, \dots, K_{\max}\}$  do
5:   detect modal peaks of  $p$ ; assign each bin to nearest peak via Voronoi ▷ init
6:   for each  $k \in \{1, \dots, K\}$  do
7:      $f_k^{(0)} \leftarrow \text{FITCOMPONENT}(p \cdot \mathbf{1}\{\text{bin} \in \text{Voronoi cell } k\})$ 
8:      $\pi_k^{(0)} \leftarrow$  prior mass in Voronoi cell  $k$ 
9:   end for
10:   $t \leftarrow 0$ ;  $\ell^{\text{best}} \leftarrow -\infty$ ; noimp  $\leftarrow 0$ 
11:  while noimp  $< \Pi$  do
12:     $F_k^{(t)}(j_0, j_1) \leftarrow \int_{B_{j_0, j_1}} f_k^{(t)}$ 
13:     $\gamma_{j_0 j_1, k}^{(t+1)} \leftarrow \pi_k^{(t)} F_k^{(t)}(j_0, j_1) / \sum_l \pi_l^{(t)} F_l^{(t)}(j_0, j_1)$ 
14:     $\pi_k^{(t+1)} \leftarrow \sum_{j_0, j_1} p_{j_0, j_1} \gamma_{j_0 j_1, k}^{(t+1)}$ 
15:     $f_k^{(t+1)} \leftarrow \text{FITCOMPONENT}(p \cdot \gamma_{\cdot, k}^{(t+1)})$ 
16:     $\ell^{(t+1)} \leftarrow \sum_{j_0, j_1} p_{j_0, j_1} \log \sum_k \pi_k^{(t+1)} F_k^{(t+1)}(j_0, j_1)$ 
17:    if  $\ell^{(t+1)} > \ell^{\text{best}}$  then
18:       $\ell^{\text{best}} \leftarrow \ell^{(t+1)}$ ; noimp  $\leftarrow 0$ 
19:    else
20:      noimp  $\leftarrow$  noimp + 1
21:    end if
22:     $t \leftarrow t + 1$ 
23:  end while
24:   $\hat{f}^{(K)} \leftarrow \sum_k \pi_k^{(t)} f_k^{(t)}$ 
25:   $\text{BIC}(K) \leftarrow -2n_{\text{eff}} \sum_{j_0, j_1} p_{j_0, j_1} \log \hat{f}^{(K)}(j_0, j_1) + (3K - 1) \log n_{\text{eff}}$  ▷ Eq. (??)
26: end for
27:  $K^* \leftarrow \text{argmin}_K \text{BIC}(K)$ 
28: return  $\hat{p}_{\text{MALC}} \leftarrow \hat{f}^{(K^*)}$ 

```

Algorithm 2 Weighted 2-Wasserstein barycenter of N densities on a common grid.

Require: Per-query densities $\{\hat{p}_i\}_{i=1}^N$ evaluated on grid $x_1 < \dots < x_M$ with spacing Δx ; weights $\alpha \in \Delta^{N-1}$ (default uniform); number of quantile levels M_τ (default 4001).

Ensure: Barycenter density p^* on the same grid.

```

1:  $\tau_j \leftarrow (j - \frac{1}{2})/M_\tau$  ▷  $j = 1, \dots, M_\tau$ 
2: for each  $i \in \{1, \dots, N\}$  do
3:    $F_i(x_k) \leftarrow \sum_{\ell \leq k} \frac{1}{2} (\hat{p}_i(x_{\ell-1}) + \hat{p}_i(x_\ell)) \Delta x$  ▷ trapezoidal CDF,  $F_i(x_0) = 0$ 
4:    $F_i \leftarrow F_i / F_i(x_M)$ 
5:    $Q_i(\tau_j) \leftarrow \text{interp}(\tau_j; F_i, x)$  ▷ invert CDF at each probability level
6: end for
7:  $Q^*(\tau_j) \leftarrow \sum_{i=1}^N \alpha_i Q_i(\tau_j)$  ▷ Agueh–Carlier closed form (??)
8:  $F^*(x_k) \leftarrow \text{interp}(x_k; Q^*, \tau)$  ▷ invert  $Q^*$  back to a CDF on the  $x$ -grid
9:  $p^*(x_k) \leftarrow (F^*(x_{k+1}) - F^*(x_{k-1})) / (2\Delta x)$  ▷ central-difference derivative
10:  $p^* \leftarrow p^* / (\Delta x \sum_k p^*(x_k))$  ▷ renormalise to integrate to 1
11: return  $p^*$ 

```
